

Codecov Test Evidence

Campus Food Ordering Platform

Project	Campus Food Ordering Platform
Evidence Type	Automated testing and code coverage evidence
Tool Used	Codecov
Branch Tested	main
Latest Source Commit Shown	4a0c149
Updated Coverage	81.42%
Covered Lines	1420 of 1744

Overall Coverage 81.42% Main branch coverage reported by Codecov.	Covered Lines 1420 / 1744 Lines covered by automated tests.	Missed Lines 324 Tracked lines not covered yet.	Coverage Trend +100.00% Positive trend shown on Codecov dashboard.
---	---	---	--

1. Purpose of this Document

This document provides updated Codecov evidence for the Campus Food Ordering Platform. It shows that automated testing is connected to the GitHub repository and that the main branch has measurable test coverage.

The screenshots included below replace the older Codecov screenshots. The latest captured project coverage is 81.42%, with 1420 of 1744 tracked lines covered.

2. Executive Summary

The updated Codecov report shows that the project has passed the 80% overall coverage target. The strongest tested areas are the vendor code and the Cloudflare API / backend logic. Authentication is also well covered. The student area has acceptable coverage, but some user-interface heavy files can still be improved in future work.

3. Overall Coverage Breakdown

The table below summarises the updated Codecov folder-level results from the latest screenshots.

Area	Tracked Lines	Covered	Missed	Coverage	Interpretation
auth	195	161	34	82.56%	Good coverage for login and registration logic.
functions/api	138	127	11	92.03%	Very strong coverage for API, analytics, and payment logic.
student	898	668	230	74.39%	Acceptable coverage; more UI paths can be tested later.
tests/mocks	18	2	16	11.11%	Low coverage is expected because mocks support tests rather than production features.
vendor	495	462	33	93.33%	Very strong coverage for vendor workflows.
Subtotal	1744	1420	324	81.42%	Overall project coverage is above 80%.

Figure 1: Codecov main branch overview

This overview confirms that Codecov is connected to the repository and that the main branch is being measured. The latest captured coverage is 81.42%, with 1420 of 1744 tracked lines covered. The dashboard also shows the latest source commit as 4a0c149 and a +100.00% trend.

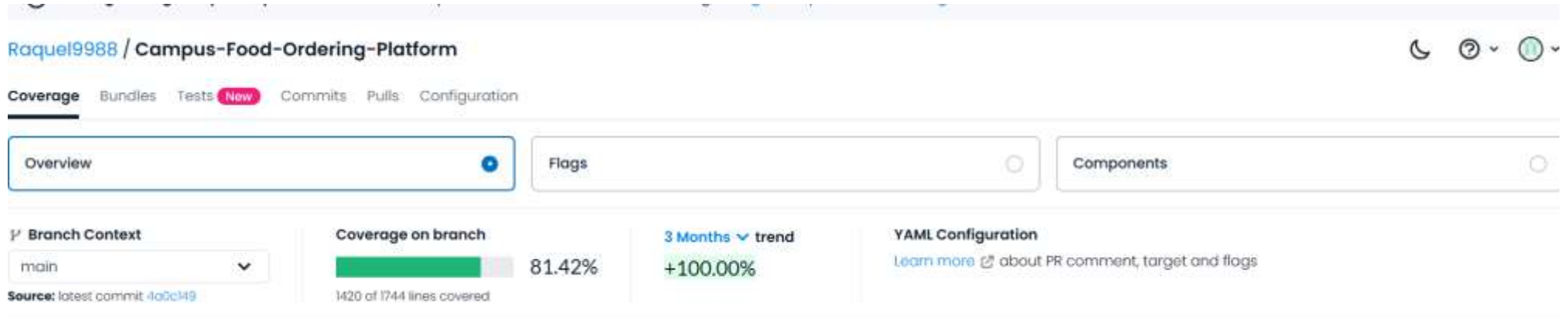


Figure 1: Codecov main branch overview.

Figure 2: Codecov coverage trend and repository coverage wheel

This screenshot gives visual evidence of the coverage trend over time. It also shows the repository-level coverage wheel, which provides a quick visual summary of covered and missed code areas.

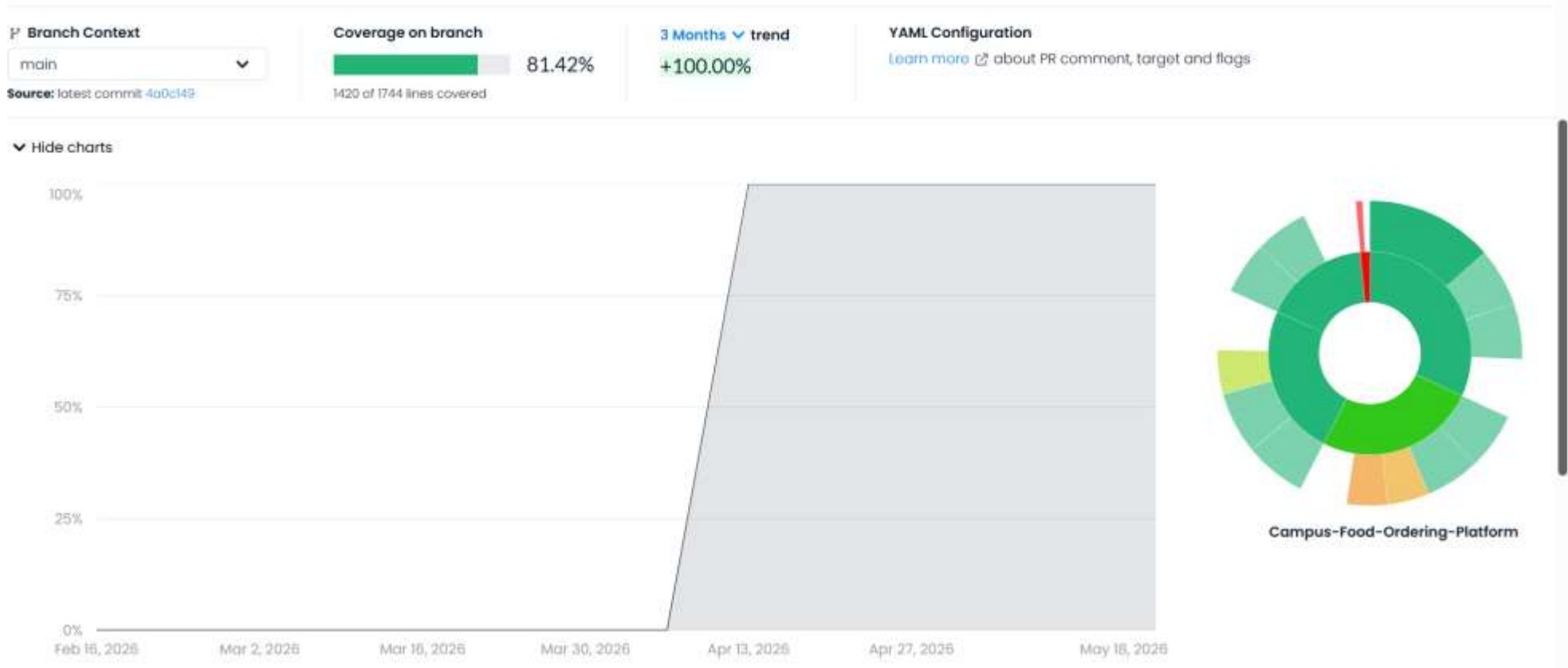


Figure 2: Codecov coverage trend and repository coverage wheel.

Figure 3: Folder-level coverage breakdown

This breakdown shows the main folders included in the coverage report. The strongest production-code coverage is in functions/api and vendor. The student folder is still acceptable but has more missed lines because it contains more interface and interaction logic. The tests/mocks folder is low because mock files support tests rather than representing production features.



Figure 3: Folder-level coverage breakdown.

Figure 4: Authentication file coverage

The authentication files have strong coverage overall. login.js and register.js are both above 82%, showing that the login and registration logic was tested as part of the automated test suite.

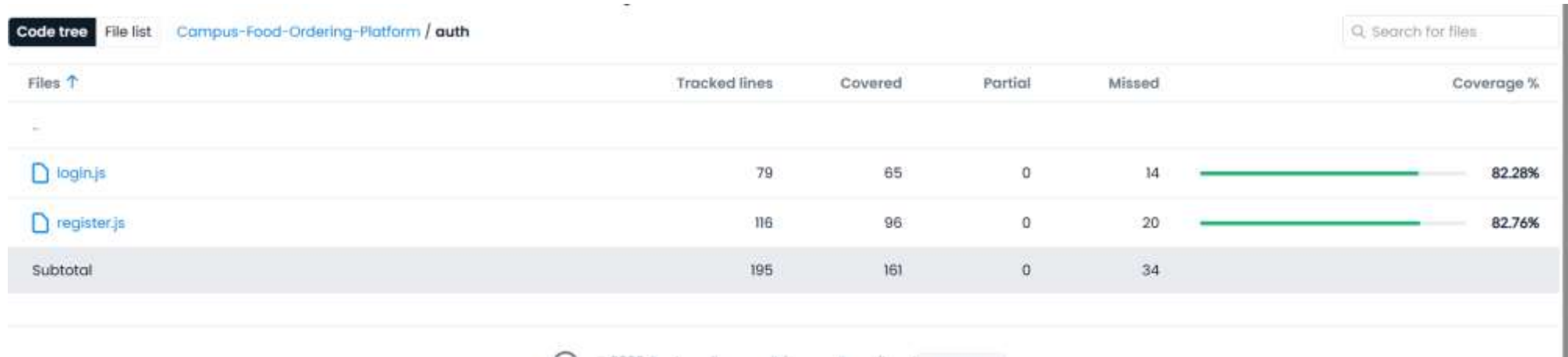


Figure 4: Authentication file coverage.

Figure 5: API and payment-related coverage

The API folder has very strong coverage. The PayFast folder is fully covered, while analytics.js and payment.js are both above 87%. This is important because payment and analytics logic are key backend features in the project.



Figure 5: API and payment-related coverage.

Figure 6: Student feature coverage

The student folder includes cart, dashboard, menu, and order-tracking logic. `student-cart.js` and `student-dashboard.js` are strongly covered, while `my-orders.js` and `student-menu.js` have lower coverage because they include more DOM rendering and user-interface paths.

The image is a screenshot of the Codecov web interface showing the test coverage for the 'student' directory. At the top, there's a navigation bar with 'Code tree', 'File list', and the path 'Campus-Food-Ordering-Platform / student'. A search bar is on the right. Below this is a table with columns: 'Files', 'Tracked lines', 'Covered', 'Partial', 'Missed', a visual progress bar, and 'Coverage %'. The table lists four files: 'my-orders.js' (65.15% coverage), 'student-cart.js' (85.98% coverage), 'student-dashboard.js' (87.88% coverage), and 'student-menu.js' (64.37% coverage). A 'Subtotal' row at the bottom shows 898 tracked lines, 668 covered, 0 partial, and 230 missed lines.

Files ↑	Tracked lines	Covered	Partial	Missed		Coverage %
 <code>my-orders.js</code>	241	157	0	84		65.15%
 <code>student-cart.js</code>	264	227	0	37		85.98%
 <code>student-dashboard.js</code>	132	116	0	16		87.88%
 <code>student-menu.js</code>	261	168	0	93		64.37%
Subtotal	898	668	0	230		

Figure 6: Student feature coverage.

Figure 7: Vendor feature coverage

The vendor folder has strong coverage overall. menuCreation.js and orders.js are both above 96%, showing strong test coverage for core vendor workflows. vendor-dashboard.js has lower coverage and is noted as an area for future improvement.



Figure 7: Vendor feature coverage.

10. Detailed Coverage Interpretation

The tables below explain the most important coverage results from the Codecov screenshots. These values help show which parts of the application were tested most strongly and which parts could be improved in future iterations.

Authentication Coverage

File / Area	Coverage	Covered Lines	Comment
login.js	82.28%	65 of 79	Login logic is well covered.
register.js	82.76%	96 of 116	Registration logic is well covered.

API / Backend Coverage

File / Area	Coverage	Covered Lines	Comment
payfast folder	100.00%	39 of 39	PayFast callback/payment support logic is fully covered.
analytics.js	87.50%	42 of 48	Analytics endpoint logic is strongly covered.
payment.js	90.20%	46 of 51	Payment API logic is strongly covered.

Student Feature Coverage

File / Area	Coverage	Covered Lines	Comment
my-orders.js	65.15%	157 of 241	Order display logic has partial coverage and can be improved.
student-cart.js	85.98%	227 of 264	Cart logic is strongly covered.
student-dashboard.js	87.88%	116 of 132	Dashboard logic is strongly covered.
student-menu.js	64.37%	168 of 261	Menu browsing and filtering has partial coverage and can be improved.

Vendor Feature Coverage

File / Area	Coverage	Covered Lines	Comment
menuCreation.js	96.25%	257 of 267	Menu creation and update logic is very strongly covered.
orders.js	97.04%	164 of 169	Vendor order workflow is very strongly covered.
vendor-dashboard.js	69.49%	41 of 59	Dashboard display logic has acceptable but improvable coverage.

11. Assessment Relevance

This evidence supports the testing and continuous integration requirements of the final submission. It shows that the project is not only implemented, but also connected to an automated coverage reporting workflow.

Requirement	Evidence in this document
Automated testing evidence	Codecov dashboard screenshots show measured test coverage on the main branch.
Coverage percentage	Overall coverage is 81.42%, which is above the 80% target.
Backend/API testing	functions/api has 92.03% coverage, including strong payment and analytics coverage.
Frontend testing	auth, student, and vendor folders show measurable coverage across main user workflows.
Updated evidence	The screenshots in this document replace the older Codecov evidence screenshots.

12. Conclusion

The updated Codecov screenshots show that automated testing and coverage reporting are active for the Campus Food Ordering Platform. The main branch currently has 81.42% coverage, with particularly strong coverage in the API/backend and vendor areas. This provides clear evidence that the team used automated tests and measured coverage as part of the final software design submission.

Note: The tests/mocks folder has low coverage because mock files are support files used by the test suite. This does not indicate weak coverage of the production application features.